

Application de modération du site R3st0.fr - Compte rendu- itération n2

Lien : https://gitlab.com/de.jesus.vizcaino/gestion_avis

Table des matières

Daily scrum 1	2
Creation dela structure du projet en une structure mvc	2
Ticket 6 correction :	4
Ticket 8	5
Ticket 4	6
Ticket 5 :	7
API pour supprimer un avis.....	9

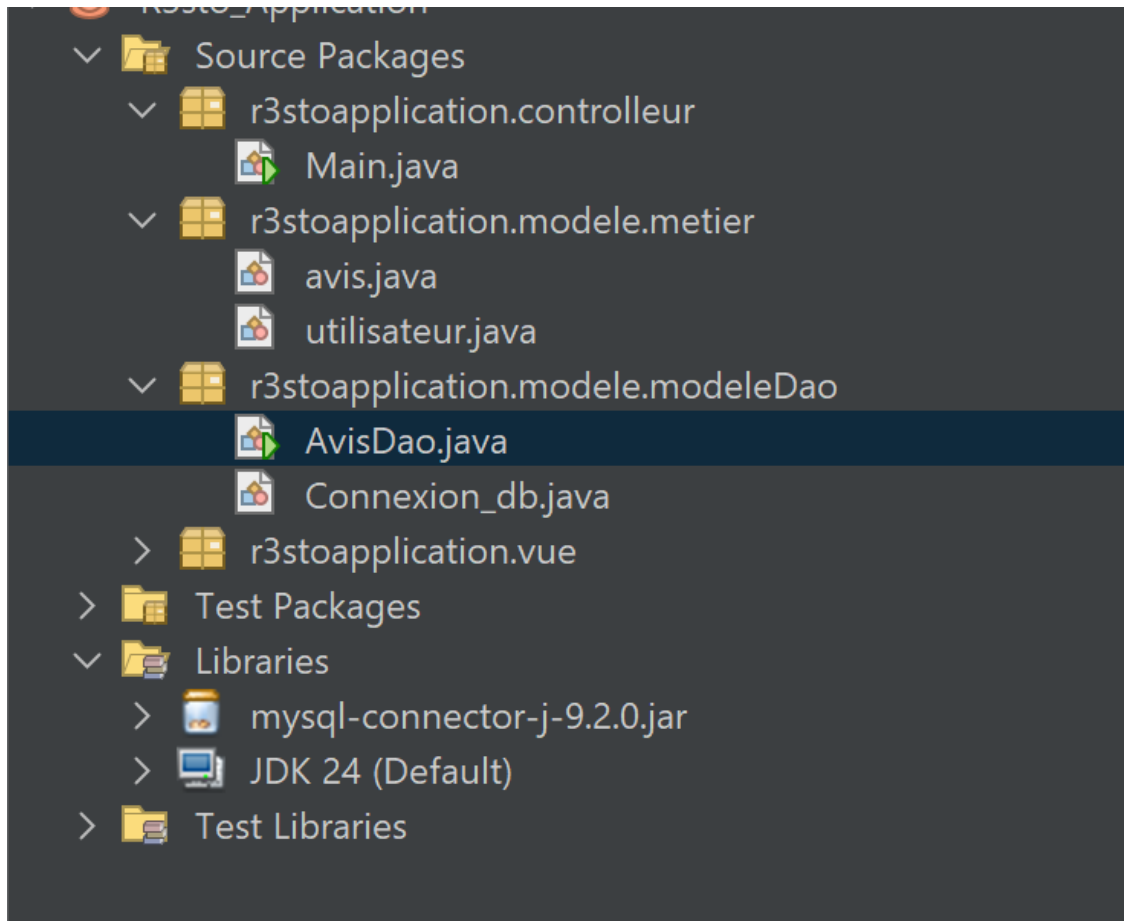
Daily scrum 1

Commencement des tickets de l'itération 2, après les tickets création d'exécutable

Rdv fin seance 1 :

- Garence : A besoin du GetAllAvis, correction de tickets fait,
- Heymad : getAllAvis masquer en cours
- Nikauly : modele mcv

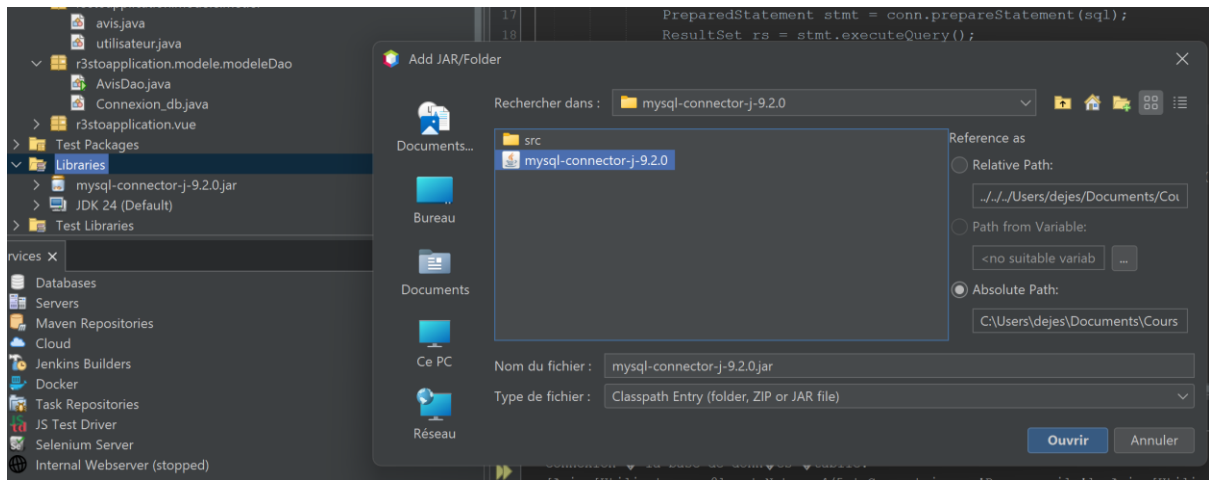
Creation dela structure du projet en une structure mvc :



Récupération des avis :

Ajout du mysql connector :

Nikauly, Heymad, Garence



Connection base de données, fichier connection db :

```
18 // Paramètres de connexion
19 private static final String URL = "jdbc:mysql://localhost:3306/resto2";
20 private static final String USER = "root";
21 private static final String PASSWORD = "";
22
23 // Instance unique (Singleton)
24 private static Connection instance = null;
25
26 /**
27  * Constructeur privé — empêche l'instanciation directe.
28  */
29 private Connexion_db() {}
30
31 /**
32  * Retourne la connexion active (ou en crée une nouvelle si nécessaire).
33  *
34  * @return Connection JDBC vers la base de données resto2
35  * @throws SQLException si la connexion échoue
36  */
37 public static Connection getConnexion() throws SQLException {
38     if (instance == null || instance.isClosed()) {
39         try {
40             Class.forName("com.mysql.cj.jdbc.Driver");
41             instance = DriverManager.getConnection(URL, USER, PASSWORD);
42             System.out.println("Connexion à la base de données établie.");
43         } catch (ClassNotFoundException e) {
44             System.err.println("Driver MySQL introuvable : " + e.getMessage());
45             throw new SQLException("Driver MySQL introuvable", e);
46         }
47     }
48     return instance;
49 }
```

Ticket 6 correction :

```
public List<avis> getAllAvisByDateDecroissante() {
    List<avis> liste = new ArrayList<>();
    // ORDER BY c.date_avis DESC permet de trier du plus récent au plus ancien
    String sql = "SELECT c.idR, c.idU, c.note, c.commentaire, u.pseudoU " +
        "FROM critiquer c JOIN utilisateur u ON c.idU = u.idU " +
        "ORDER BY c.date_avis DESC;";

    try {
        Connection conn = Connexion_db.getConnexion();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            avis unAvis = new avis();
            unAvis.setNote(rs.getInt("note"));
            unAvis.setCommentaire(rs.getString("commentaire"));
            unAvis.setIdU(rs.getInt("idU"));

            utilisateur unUtilisateur = new utilisateur(rs.getString("pseudoU"));
            unAvis.setAuteur(unUtilisateur);

            liste.add(unAvis);
        }
    } catch (SQLException e) {
        System.err.println("Erreur SQL (tri décroissant) : " + e.getMessage());
    }
    return liste;
}

public List<avis> getAllByJour(java.sql.Date jourRecherche) {
```

Methode décroissante :

```
public List<avis> getAllAvisByDateDecroissante() {
    List<avis> liste = new ArrayList<>();
    // ORDER BY c.date_avis DESC permet de trier du plus récent au plus ancien
    String sql = "SELECT c.idR, c.idU, c.note, c.commentaire, u.pseudoU " +
        "FROM critiquer c JOIN utilisateur u ON c.idU = u.idU " +
        "ORDER BY c.date_avis DESC;";

    try {
        Connection conn = Connexion_db.getConnexion();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            avis unAvis = new avis();
            unAvis.setNote(rs.getInt("note"));
            unAvis.setCommentaire(rs.getString("commentaire"));
            unAvis.setIdU(rs.getInt("idU"));

            utilisateur unUtilisateur = new utilisateur(rs.getString("pseudoU"));
            unAvis.setAuteur(unUtilisateur);

            liste.add(unAvis);
        }
    } catch (SQLException e) {
        System.err.println("Erreur SQL (tri décroissant) : " + e.getMessage());
    }
    return liste;
}

public List<avis> getAllByJour(java.sql.Date jourRecherche) {
```

Modification de la base de donnée pour avoir la date :

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0390 seconde(s).)

```
ALTER TABLE critiqueur ADD COLUMN date_avis DATETIME DEFAULT CURRENT_TIMESTAMP;
```

[\[Éditer en ligne \]](#) [\[Éditer \]](#) [\[Créer le code source PHP \]](#)

Methode getAllByJour

```
public List<avis> getAllByJour(java.sql.Date jourRecherche) {
    List<avis> listeByJour = new ArrayList<>();
    // DATE(c.date_avis) permet d'ignorer l'heure pour comparer uniquement la date
    String sql = "SELECT c.idR, c.idU, c.note, c.commentaire, u.pseudoU " +
        "FROM critiqueur c JOIN utilisateur u ON c.idU = u.idU " +
        "WHERE DATE(c.date_avis) = ?;";

    try {
        Connection conn = Connexion_db.getConnexion();
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setDate(1, jourRecherche);

        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            avis unAvis = new avis();
            unAvis.setNote(rs.getInt("note"));
            unAvis.setCommentaire(rs.getString("commentaire"));
            unAvis.setIdU(rs.getInt("idU"));

            utilisateur unUtilisateur = new utilisateur(rs.getString("pseudoU"));
            unAvis.setAuteur(unUtilisateur);

            listeByJour.add(unAvis);
        }
    } catch (SQLException e) {
        System.err.println("Erreur SQL (par jour) : " + e.getMessage());
    }
    return listeByJour;
}
```

Ticket 8

Methode getAllBySemaine

```
public List<avis> getAllBySemaine(java.sql.Date dateDansLaSemaine) {
    List<avis> listeBySemaine = new ArrayList<>();
    // YEARWEEK compare l'année et la semaine. Le '1' signifie "la semaine commence le lundi"
    String sql = "SELECT c.idR, c.idU, c.note, c.commentaire, u.pseudoU " +
        "FROM critiquer c JOIN utilisateur u ON c.idU = u.idU " +
        "WHERE YEARWEEK(c.date_avis, 1) = YEARWEEK(?, 1)";

    try {
        Connection conn = Connexion_db.getConnexion();
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setDate(1, dateDansLaSemaine);

        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            avis unAvis = new avis();
            unAvis.setNote(rs.getInt("note"));
            unAvis.setCommentaire(rs.getString("commentaire"));
            unAvis.setIdU(rs.getInt("idU"));

            utilisateur unUtilisateur = new utilisateur(rs.getString("pseudoU"));
            unAvis.setAuteur(unUtilisateur);

            listeBySemaine.add(unAvis);
        }
    } catch (SQLException e) {
        System.err.println("Erreur SQL (par semaine) : " + e.getMessage());
    }
    return listeBySemaine;
}
```

Ticket 4

Methode GetAllAvis

```
public List<avis> getAllAvis() {
    List<avis> liste = new ArrayList<>();
    String sql = "SELECT c.idR, c.idU, c.note, c.commentaire, u.pseudoU FROM critiquer c join utilisateur u on c.idU = u.idU";

    try {
        Connection conn = Connexion_db.getConnexion();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            avis unAvis = new avis();
            unAvis.setNote(rs.getInt("note"));
            unAvis.setCommentaire(rs.getString("commentaire"));
            unAvis.setIdU(rs.getInt("idU"));
            utilisateur unUtilisateur = new utilisateur(rs.getString("pseudoU"));
            unAvis.setAuteur(unUtilisateur);

            liste.add(unAvis);
        }
    } catch (SQLException e) {
        System.err.println("Erreur SQL : " + e.getMessage());
    }
    return liste;
}

public List<avis> getAllByJour(java.sql.Date jourRecherche) {
    List<avis> listeByJour = new ArrayList<>();
    String requetteByJour = "SELECT idR, note, commentaire, idU FROM critiquer WHERE date_avis = ?";
}
```

Ticket 5 :

Note	Commentaire	Utilisateur
4	Bon accueil.	@lex
3	moyen	drskott
2	bof.	drskott
3	Très bonne entrecote, l...	pich
1	À éviter...	pich
5		pich
0		pich
4	Très bon accueil :)	pich
4	Très bon accueil.	Nico40
1	Cuisine tres moyenne.	Nico40
5	Rapide	Nico40

```

29 private void chargerAvis() {
30     String[] colonnes = {"Note", "Commentaire", "Utilisateur"};
31
32     DefaultTableModel model = new DefaultTableModel(colonnes, 0);
33
34     AvisDao dao = new AvisDao();
35     List<avis> liste = dao.getAllAvis();
36
37     for (avis a : liste) {
38         Object[] row = {
39             a.getNote(),
40             a.getCommentaire(),
41             a.getPseudo()
42         };
43         model.addRow(row);
44     }
45
46     tableAvis = new javax.swing.JTable(model);
47     jScrollPane1.setViewportView(tableAvis);
48 }

```

Affichage des avis masqués par le responsable du restaurant

```
49  /**
50   * Requête SQL : sélectionne tous les avis masqués (masque = 1) d'un restaurant,
51   * mais seulement si l'utilisateur en paramètre est le responsable de ce resto.
52   */
53  private static final String REQUETE_ALL_AVIS_MASQUER =
54      "SELECT " +
55      "    c.idR, " +
56      "    r.nomR AS nom_restaurant, " +
57      "    c.idU, " +
58      "    u.pseudoU AS pseudo_utilisateur, " +
59      "    c.note, " +
60      "    c.commentaire, " +
61      "    c.date_avis " +
62      "FROM critiquer c " +
63      "JOIN resto r ON c.idR = r.idR " +
64      "JOIN utilisateur u ON c.idU = u.idU " +
65      "WHERE c.masque = 1 AND r.idU_responsable = ? " +
66      "ORDER BY c.idR ASC";
67
```

```
68  /**
69   * Récupère tous les avis masqués pour les restaurants dont le responsable est fourni.
70   *
71   * @param idResponsable L'ID de l'utilisateur faisant la demande (le responsable)
72   * @return Liste de tableaux d'objets représentant chaque avis masqué
73   * @throws SQLException si la requête échoue
74   */
75  public static List<Object[]> getAllAvisMasquer(long idResponsable) throws SQLException {
76      List<Object[]> listeAvis = new ArrayList<>();
77
78      Connection connexion = Connexion_db.getConnexion();
79      try (PreparedStatement stmt = connexion.prepareStatement(REQUETE_ALL_AVIS_MASQUER)) {
80          // Injection de l'identifiant du responsable de façon sécurisée
81          stmt.setLong(1, idResponsable);
82
83          try (ResultSet rs = stmt.executeQuery()) {
84              while (rs.next()) {
85                  Object[] avis = new Object[]{
86                      rs.getLong("idR"),
87                      rs.getString("nom_restaurant"),
88                      rs.getLong("idU"),
89                      rs.getString("pseudo_utilisateur"),
90                      rs.getObject("note"),
91                      rs.getString("commentaire"),
92                      rs.getTimestamp("date_avis")
93                  };
94                  listeAvis.add(avis);
95              }
96          }
97      }
98
99      return listeAvis;
100  }
101
102  }
103
```

La méthode `getAllAvisMasquer` permet de récupérer tous les avis masqués d'un restaurant, mais uniquement si l'utilisateur qui fait la demande est bien le responsable du restaurant. Pour cela, une requête SQL sélectionne tous les avis dont le champ

masque vaut 1, puis vérifie que l'ID du responsable correspond à celui passé en paramètre.

API pour supprimer un avis

```
15  */
16  public class DeleteAvis {
17
18      /**
19       * Requête SQL : supprime la ligne de la table critiquer
20       * correspondant à la clé primaire composée idR (restaurant) et idU (utilisateur).
21       */
22      private static final String REQUETE_SUPPRIMER_AVIS =
23          "DELETE FROM critiquer WHERE idR = ? AND idU = ?";
24
25      /**
26       * Supprime un avis de la base de données.
27       *
28       * @param idR L'ID du restaurant
29       * @param idU L'ID de l'utilisateur ayant écrit l'avis
30       * @return true si l'avis a bien été supprimé, false s'il n'existait pas
31       * @throws SQLException si la requête échoue
32       */
33      public static boolean deleteAvis(long idR, long idU) throws SQLException {
34          boolean aEteSupprime = false;
35
36          Connection connexion = Connexion_db.getConnexion();
37
38          try (PreparedStatement stmt = connexion.prepareStatement(REQUETE_SUPPRIMER_AVIS)) {
39              // Assignation des paramètres
40              stmt.setLong(1, idR);
41              stmt.setLong(2, idU);
42
43              // Exécution : executeUpdate retourne le nombre de lignes supprimées
44              int lignesAffectees = stmt.executeUpdate();
45
46              // On vérifie si une ligne a effectivement été effacée
47              aEteSupprime = (lignesAffectees > 0);
48          }
49
50          return aEteSupprime;
51      }
52  }
```

La méthode `deleteAvis` permet de supprimer un avis dans la base de données en utilisant l'identifiant du restaurant et celui de l'utilisateur. Elle prépare une requête SQL sécurisée grâce à un `PreparedStatement`, puis remplace les paramètres par les valeurs reçues.